

Victoria Smith

Due Date: August 20, 2026

Java Individual Contribution Report

Assigned Features

Application Structure and Database Configuration

I established the initial project structure and PostgreSQL connection and configuration, including the initial database schema. I also created the application.properties file and configured it to be ignored by Git, allowing us to keep sensitive database credentials separate from the application source code.

User Layer and Authentication

My work on the User layer of the application included the User entity, UserDAO, and UserService classes. This included the database operations and service logic required to create and retrieve users.

I also developed the user registration and login authentication functionality, using BCrypt to hash user passwords before they were stored in the database and to validate passwords during login. After a successful login, the user's role is used to direct them to the appropriate role-based menu and restrict access to any functionality that is outside of their permissions.

Merchandise Layer

For the Merchandise layer, I created the Merchandise entity, MerchandiseDAO, and MerchandiseService classes. This included defining merchandise data and implementing the database and service-layer functionality needed to manage merchandise records. I also added the merchandise table to the PostgreSQL schema and ensured that the Java classes integrated correctly with the database.

Main Application and Role-Based Flow

I developed the Main application logic and role-based flow that connects user authentication to the rest of the app. After a successful login, the app identifies the user's role and directs them to the appropriate menu. This ensured that each user was presented with the actions available to their role and provided the flow between authentication and the role-based menus.

Login and Admin Menus

My menu contributions included the LoginMenu and AdminMenu classes. The Login menu handles user registration and authentication, including validating user input and displaying appropriate errors when login or registration attempts are unsuccessful. The Admin menu provides admins access to their role-specific actions, including viewing and deleting users, viewing membership revenue, and managing merchandise and workout classes.

Final Refactoring, Testing and Integration

I completed the final refactoring and integration pass for the app to improve consistency, readability, and overall code quality before submission. This included reviewing the completed features together, testing, cleaning up duplicated or unnecessary code, standardizing naming and structure where needed, and resolving integration issues that appeared as the different branches were merged into main. I also resolved a merge conflict during the final cleanup process and verified that the completed app still functioned correctly afterwards.

GitHub Contributions

Development was completed using separate feature branches and pull requests to keep changes organized throughout the project. I configured the main branch as a protected branch, requiring changes to be submitted through pull requests so that team members could review each other's work before it was merged into the main codebase.

- [Link to my branches](#)
- [Link to my pull requests](#)
- [Link to my commit history](#)

Challenges and Problem Solving

One of the biggest challenges I encountered at the beginning of the project was adjusting to the modular structure of a larger Java application. Our previous projects had all been smaller in scope, so it initially took some time to understand how the entity, DAO, service, and menu classes needed to work together as a complete application. Working in IntelliJ for the first time also added to the learning curve, but ultimately helped me in the end.

As the project progressed, I became much more comfortable tracing the flow of data through the different layers and understanding the responsibility of each class. Breaking features into smaller pieces and testing them individually before integrating them into the larger application helped make the structure much easier to manage. By the end of the project, the separation between database, DAO, service, and menu layers made the application easier to understand and refactor rather than feeling like added complexity.

Skills Learned

This project gave me a much stronger understanding of how a larger Java app can be structured using separate, modular class layers. I became comfortable working with JDBC and PostgreSQL in a larger application and I feel like I've gained practical experience with implementing BCrypt authentication and role-based access control. I also gained experience using Maven to manage project dependencies and configuration within a larger Java project.

Team Reflection

Overall, I think our approach to dividing the application into separate features worked well for this project. Using feature branches and pull requests allowed us to work independently while still reviewing each other's work before it was merged into main. Having a protected main branch also helped prevent unfinished or conflicting changes from being added directly to the working version of the application.

For future projects, I think we could benefit from spending a bit more time at the beginning mapping out how the different features will connect before development begins. Because this was our first time working together on a much larger Java application, some of that became clearer naturally as the project progressed. Having that experience now would make it easier to plan the structure and divide the work from the beginning on a similar project in the future.

AI Usage Log

I used ChatGPT throughout the project as a learning and troubleshooting tool. I used it to help clarify unfamiliar Java concepts, work through frustrating errors, review portions of my code, and better understand how different parts of the application should interact. I also used AI to help research concepts that were new to us for this particular project, rather than just using generated code without understanding how it worked.

AI Feature Challenge Integration and Analysis

AI Dialogue

The file export feature was primarily implemented by my project partner, so my use of AI for this feature focused on reviewing and integrating the completed functionality rather than researching the initial file I/O implementation. During the final review, I used ChatGPT to help reason through exception handling within the exporter methods, specifically how SQLException and IOException should bubble up to the appropriate level of the app rather than being handled in a way that could allow a false success message to be displayed to the user.

- **Prompt:** Should SQLException and IOException be handled inside the exporter method, or bubbled up so the menu can determine whether the export actually succeeded?

Technical Reflection

Closing file streams is important because it releases system resources and ensures that buffered data is properly written to the file. If a program crashes before a stream is properly closed, data may not be fully written, and the resulting file could be incomplete. Using try with resources helps prevent this by automatically closing the stream when the operation is complete.

Code Showcase

```
package com.gymmembership.reports;

import com.gymmembership.merchandise.Merchandise;
import com.gymmembership.merchandise.MerchandiseService;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.sql.SQLException;
import java.util.ArrayList;

public class MerchandiseReportExporter {

    private final MerchandiseService merchandiseService =
        new MerchandiseService();

    private void createReportsFolder() {
        File reportsFolder = new File("reports");

        if (!reportsFolder.exists()) {
            reportsFolder.mkdir();
        }
    }

    public void exportReport() throws SQLException, IOException {
        createReportsFolder();
    }
}
```

```

        ArrayList<Merchandise> merchandise =
            merchandiseService.browseMerchandise();

        try (FileWriter writer =
            new FileWriter("reports/merchandise_report.txt")) {

            writer.write("Merchandise Inventory Report\n");
            writer.write("=====\n");

            double totalInventoryValue = 0;

            for (Merchandise item : merchandise) {

                double itemValue =
                    item.getPrice() * item.getStockLevel();

                writer.write("Merchandise ID: " + item.getMerchandiseID() +
"\n");
                writer.write("Product Name: " + item.getProductName() +
"\n");
                writer.write("Type: " + item.getType() + "\n");
                writer.write(String.format("Price: $%.2f\n",
item.getPrice()));
                writer.write("Stock Level: " + item.getStockLevel() + "\n");
                writer.write(String.format("Item Value: $%.2f\n",
itemValue));
                writer.write("-----\n");

                totalInventoryValue += itemValue;
            }

            writer.write("\n");
            writer.write(String.format("Total Inventory Value: $%.2f\n",
totalInventoryValue));
        }
    }
}

```